JAVA UNIT-4 and UNIT-5 MATERIAL

Ch. No.	Topic	Details	Marks
4 and 5	JavaFx basics and Event Driven Programming and animations	• Basic Structure of JavaFx Program • Panes • UI Controls and Shapes • Color and Font Class • Image and Image-View Class • Layout panes and shapes • Events and Events Sources • Registering Handlers and Handling events • Inner Classes, anonymous inner class handlers • Mouse and Key Events • Listeners for observable objects • Animation	14

✅ What is JavaFX?

JavaFX is a **Java library** used to create **GUI (Graphical User Interface) applications**.

👉 In simple words:
It helps you make apps with **buttons, windows, graphics, animations, and media**.

✓ It is used to:

- Design UI (user interface)
- Create graphics applications
- Build modern desktop apps

✓ It works on different platforms (Windows, Mac, Linux)

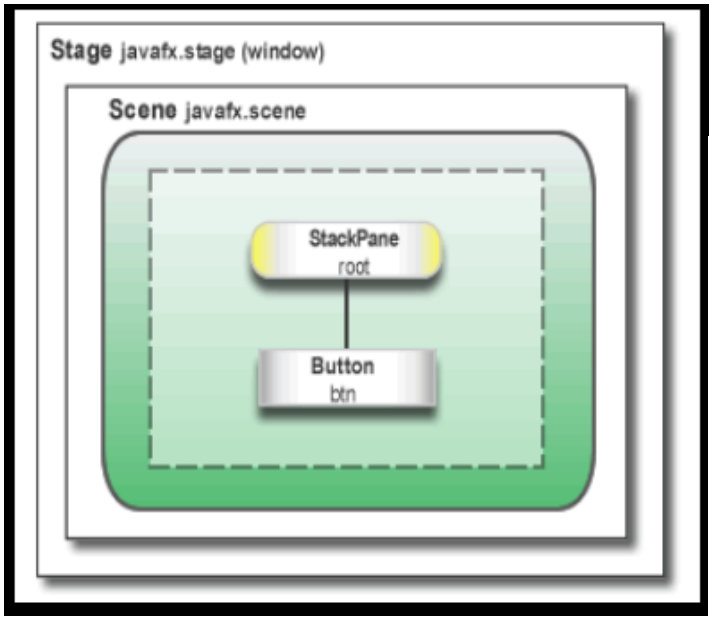
✅ Basic Structure of JavaFX Program

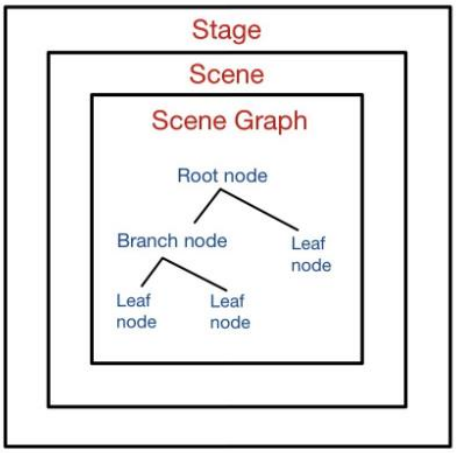
JavaFX mainly has **3 important parts**:

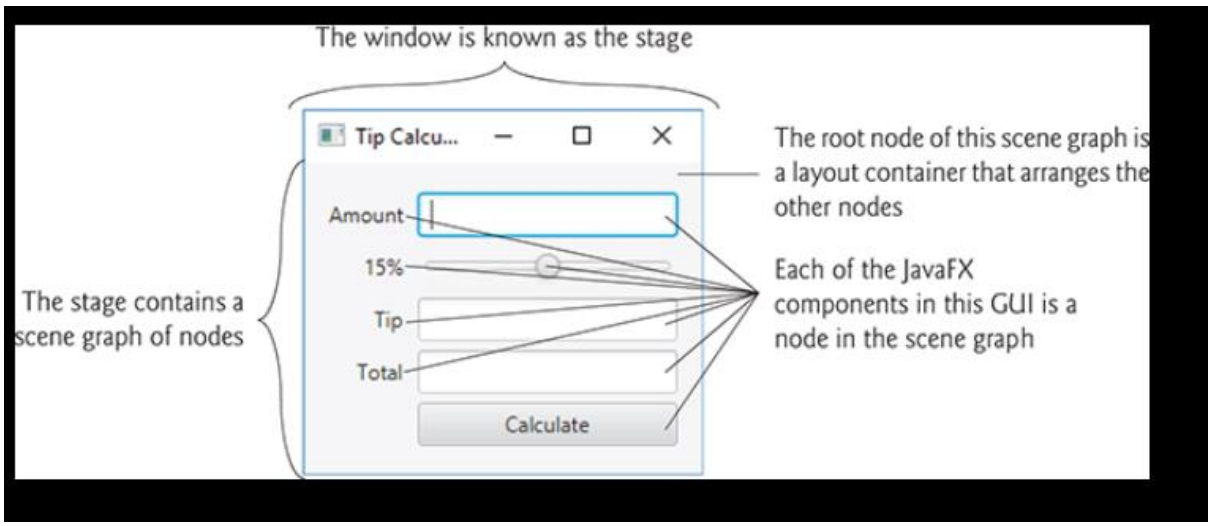
👉 Stage, Scene, Node

---

1) Stage (Window)







✓ Stage is like a **window** of the application

✓ It is the **main container** that holds everything

👉 Important points:

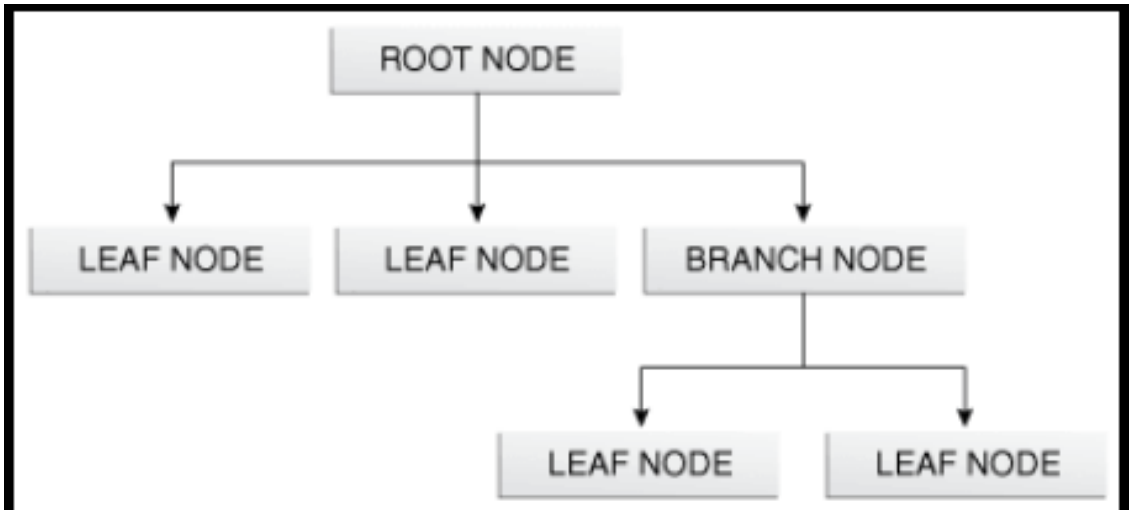
- Created automatically by JavaFX
- Passed to start() method
- Comes from javafx.stage.Stage package
- Divided into:

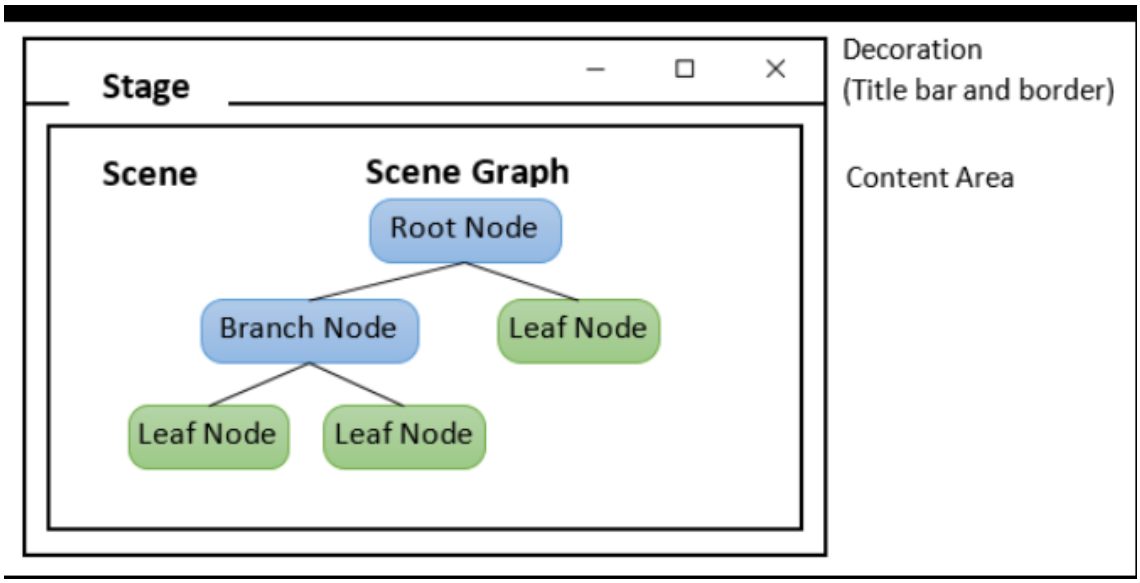
- Content area
- Border

- To display it → use show()

---

2) Scene (Content Area)





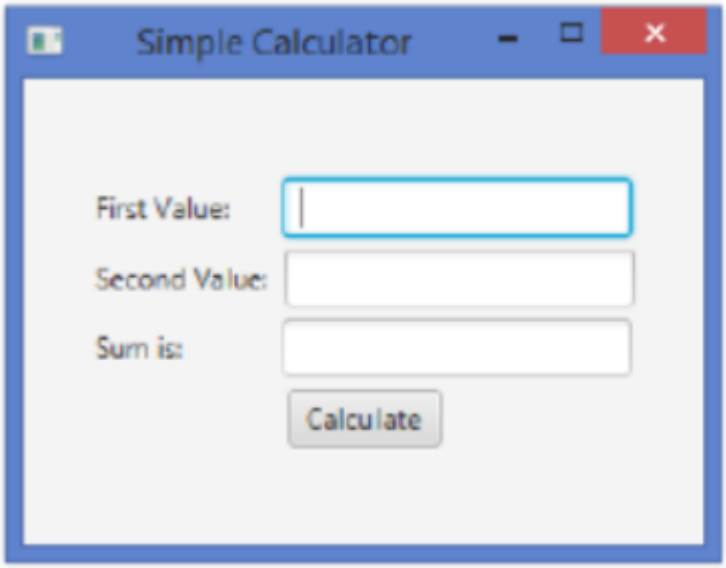
✓ Scene is the **content inside the window**

👉 Simple understanding:
Stage = Window
Scene = What is inside the window

👉 Important points:

- Contains all UI elements (buttons, text, etc.)
- Represented by javafx.scene.Scene
- One scene → only one stage at a time
- Created using new Scene()

3) Node (UI Elements)



✓ Node means **UI components**

👉 Examples:

- Button
- TextField
- Label

👉 Simple understanding:
Node = Small elements inside Scene

✅ **Important Methods in JavaFX**

1) start()

✓ Main method of JavaFX
✓ All code is written here
✓ Called automatically

---

2) init()

✓ Empty method (optional)
✓ Used for initialization
❌ Cannot create Stage or Scene here

---

3) stop()

✓ Called when application closes
✓ Used to stop or clean resources

---

4) launch()

✓ Used to start JavaFX application
✓ It is **static method**
✓ Called inside main() method

✅ **Important Rule**

👉 Every JavaFX class must extend:

```
javafx.application.Application
```

✅ Example Program (Corrected + Simple)

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;

public class A extends Application
{
    public void start(Stage ps)
    {
        Button b = new Button("OK");

        Scene s = new Scene(b, 200, 250);

        ps.setTitle("My First Program");
        ps.setScene(s);
        ps.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }

}
```

🧠 Final Memory Trick (Very Easy)

👉 Remember like this:

- **Stage = Window**
- **Scene = Content**
- **Node = Items (Button, Text)**

👉 Flow:

```
Application → Stage → Scene → Node
```

Q-2 Explain panes in JavaFx

Ans:

👉 A pane is a UI element ("Node") that contains other UI elements ("child nodes") and manages the layout of those nodes within the Pane.

👉 Following are the types of pane in JavaFx class

o Flow Pane

o Stack Pane

o Border Pane

o Grid Pane

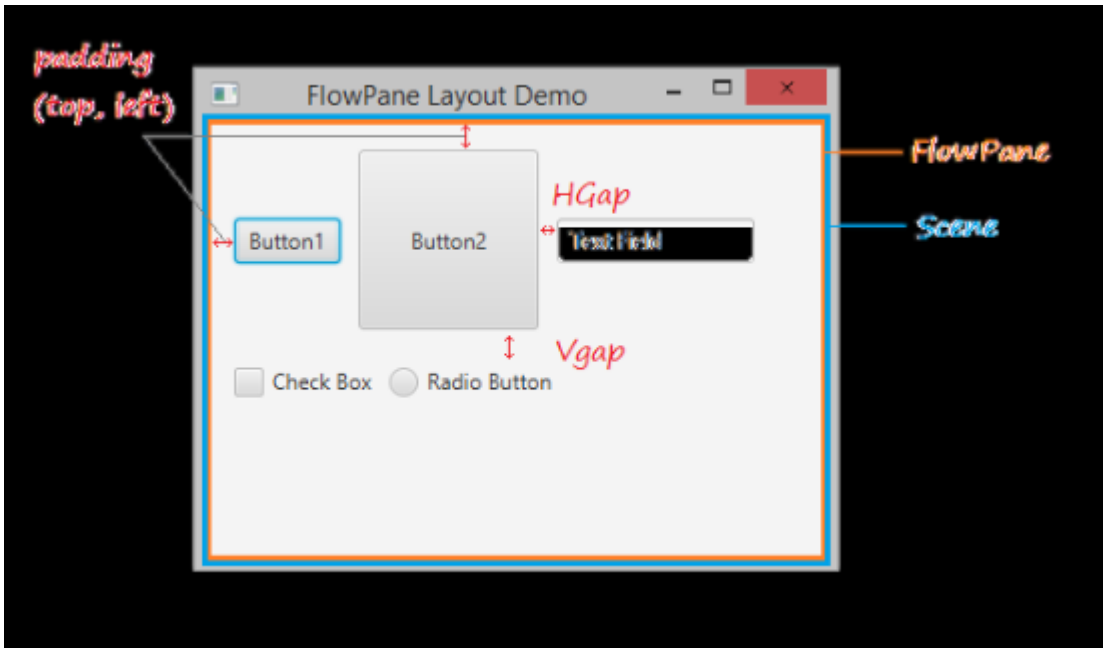
o Tile Pane

o Anchor Pane

o V-Box

o H-Box

✅ **1) Flow Pane**



🔷 **Point**

Flow Pane lays all nodes one after another in the order they were added

🔷 Simple Explanation

It puts items **one by one in a line** (left to right)

🔷 Meaning

Items follow a **flow (line)**
If space ends → go to next line

🔷 Example

Like people standing in a **queue**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage ps)
    {
        FlowPane p = new FlowPane();

        Button b1 = new Button("Ok");
        Button b2 = new Button("Cancel");

        p.getChildren().addAll(b2, b1);

        Scene s = new Scene(p, 200, 250);
        ps.setTitle("Flow layout");
        ps.setScene(s);
        ps.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

🧠 Memory Trick

👉 **Flow = Line → → → ↓**

2) Stack Pane :-

🔷 **Point**

Stack Pane lays all nodes one on top of another

🔷 **Simple Explanation**

Items are placed **over each other**

🔷 **Meaning**

Last item = **top visible**

🔷 **Example**

Like **stack of books**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;

public class A extends Application
{
    public void start(Stage ps)
    {
        StackPane p = new StackPane();

        Button b1 = new Button("Ok");
        Button b2 = new Button("Cancel");

        p.getChildren().addAll(b2, b1);

        Scene s = new Scene(p, 200, 250);
        ps.setTitle("Stack layout");
        ps.setScene(s);
        ps.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

3) Grid Pane :-

🔷 **Point**

Grid Pane creates rows and columns and places items exactly

✓ You can place elements at exact position

🔷 **Simple Explanation**

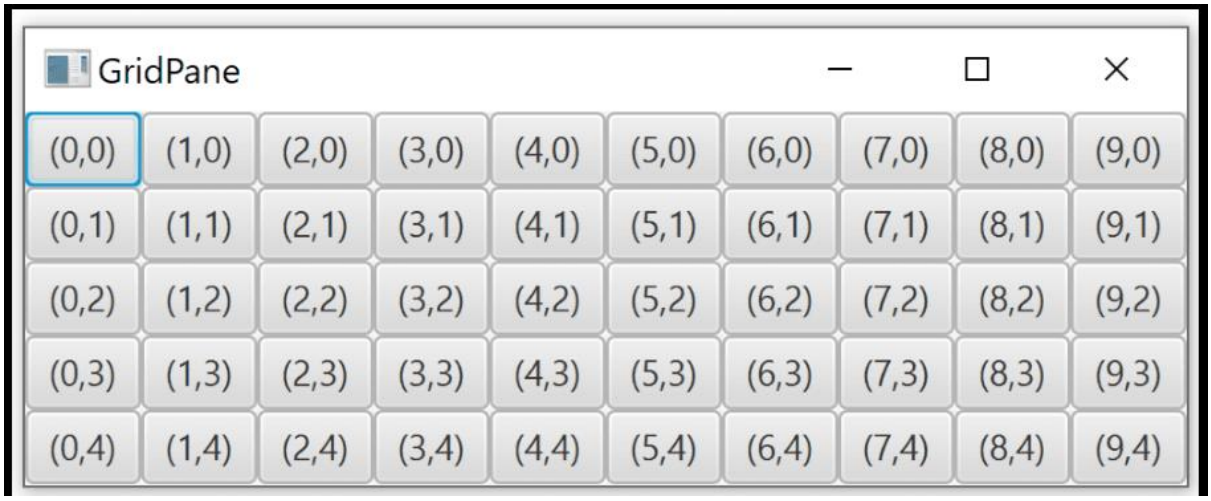
It works like a **table**

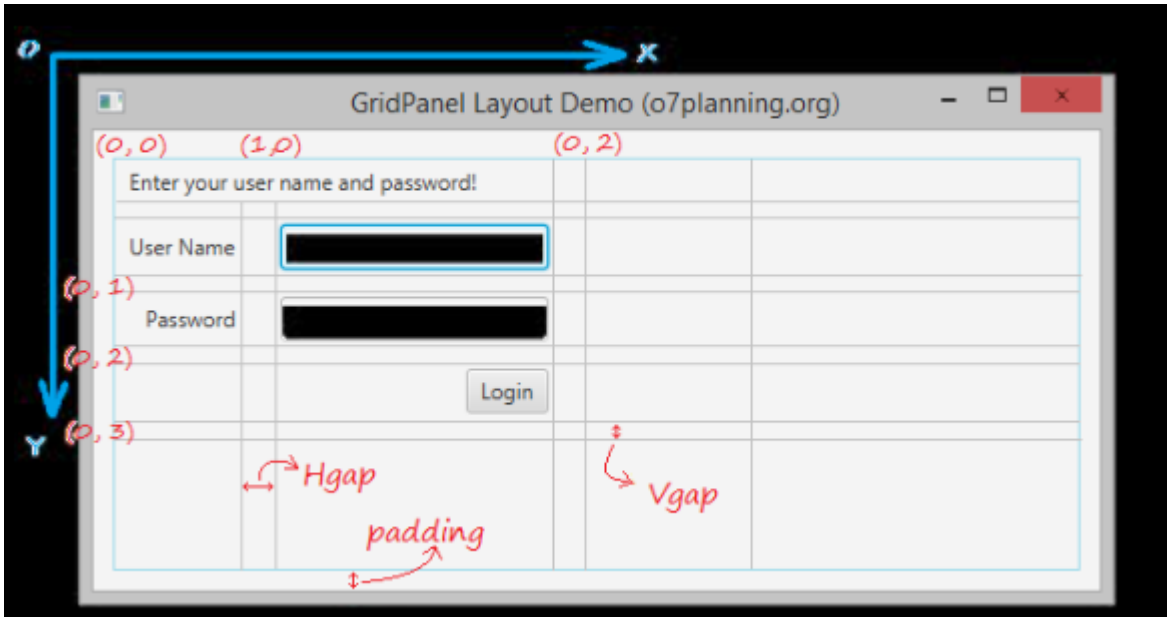
🔷 **Meaning**

You control position using **row + column**

🔷 **Example**

Like **Excel sheet**

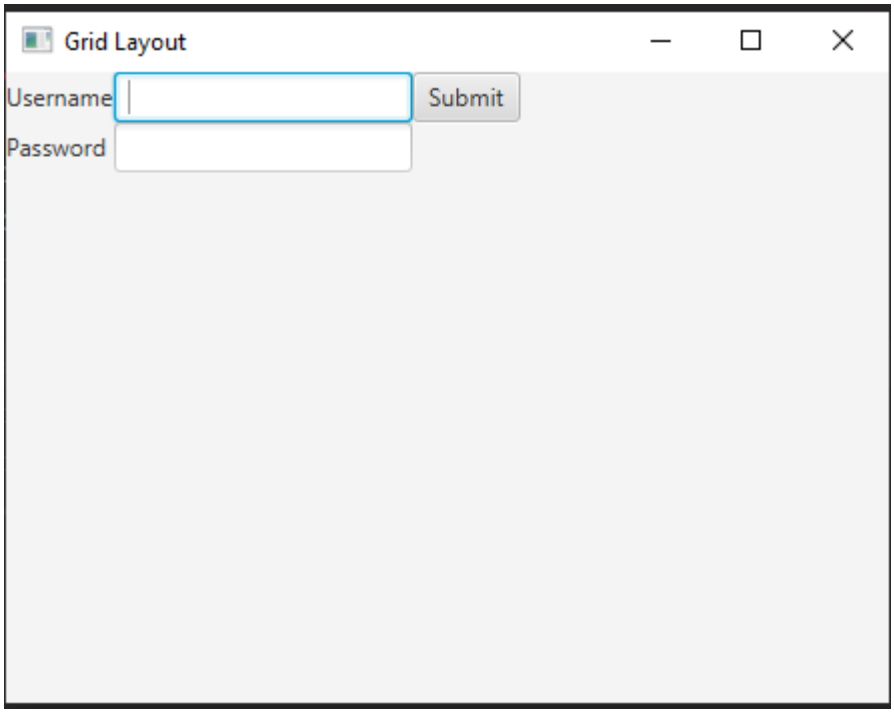





```
package JavaFx;

import javafx.application.Application;
```

```java
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;

public class vinay_7_sample extends Application
{
    @Override
    public void start(Stage primaryStage)
    {
        Label l1 = new Label("Username");
        Label l2 = new Label("Password");

        TextField t1 = new TextField();
        TextField t2 = new TextField();

        Button b = new Button("Submit");

        GridPane g = new GridPane();

        g.addRow(0, l1, t1);
        g.addRow(1, l2, t2);
        g.addRow(0, b);

        Scene s = new Scene(g, 200, 250);
        primaryStage.setTitle("Grid Layout");
        primaryStage.setScene(s);
        primaryStage.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

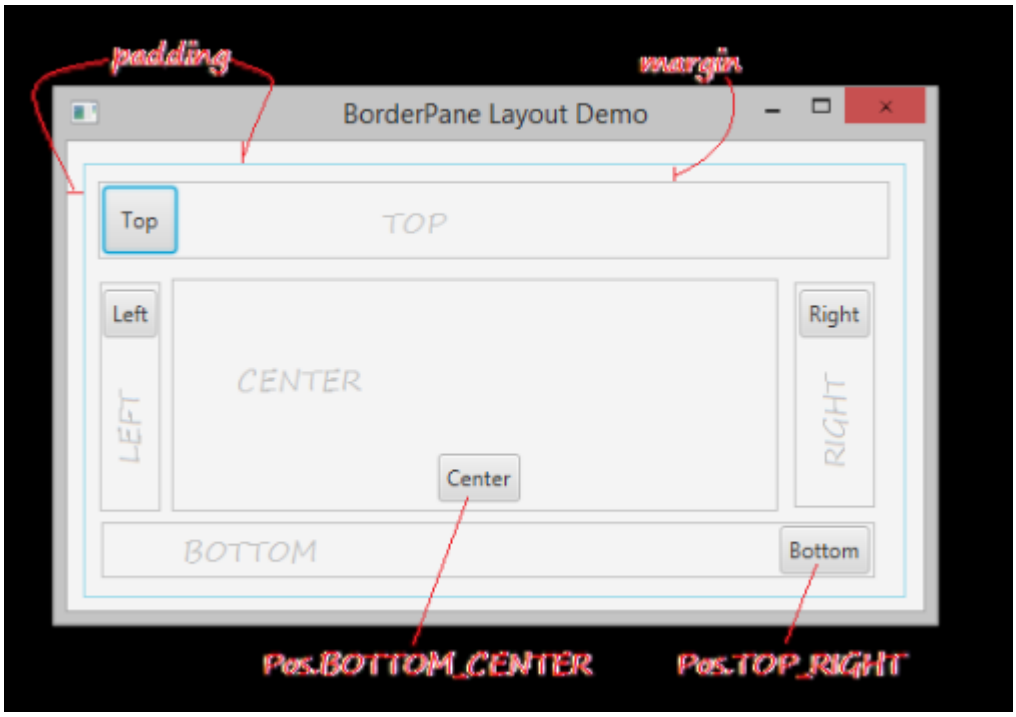
Output:



🧠 **Memory Trick**

👉 **Grid = Table (rows + columns)**

4) Border Pane



✓ BorderPane divides screen into **5 parts**:

- Top
- Bottom
- Left
- Right
- Center

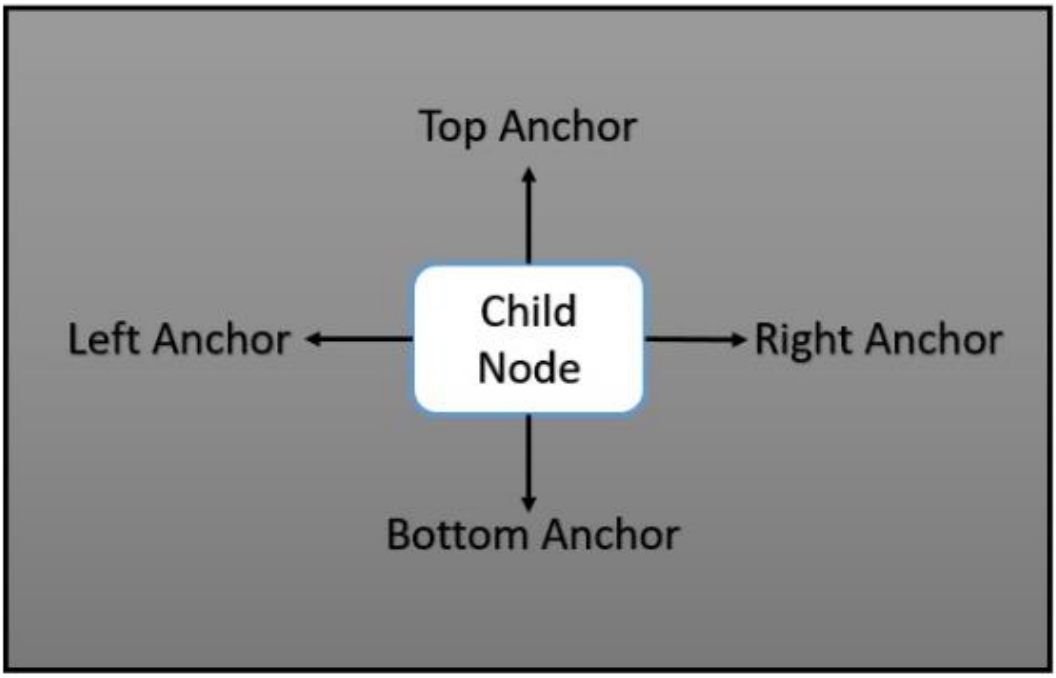
👉 You can add different nodes in each area

✓ Correct Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.BorderPane;
```

```java
public class A extends Application
{
    public void start(Stage ps)
    {
        Button b1 = new Button("Left Button");
        Button b2 = new Button("Right Button");
        Button b3 = new Button("Top Button");
        Button b4 = new Button("Bottom Button");
        Button b5 = new Button("Center Button");

        BorderPane bp = new BorderPane();

        bp.setLeft(b1);
        bp.setRight(b2);
        bp.setTop(b3);
        bp.setBottom(b4);
        bp.setCenter(b5);

        Scene s1 = new Scene(bp, 200, 250);
        ps.setScene(s1);
        ps.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

5) Anchor Pane:



✓ AnchorPane allows you to **fix position using distance (pixels)**

✓ 4 anchors:

- Top
- Bottom
- Left
- Right

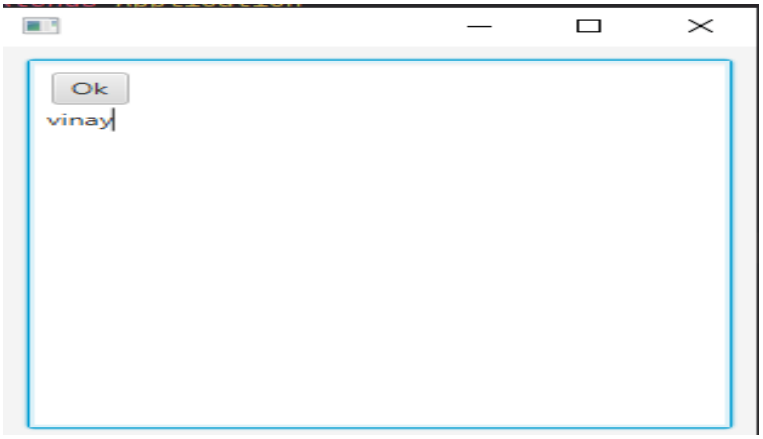
👉 You set distance from edges

✓ **Correct Example:**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.*;
import javafx.scene.layout.AnchorPane;

public class vinay_7_s extends Application
{
    public void start(Stage ps)
    {
        Button b = new Button("Ok");
        TextArea t = new TextArea();

        AnchorPane a = new AnchorPane();

        a.getChildren().addAll(t, b);
```

```
        AnchorPane.setTopAnchor(t, 10.0);
        AnchorPane.setBottomAnchor(t, 10.0);
        AnchorPane.setLeftAnchor(t, 10.0);
        AnchorPane.setRightAnchor(t, 10.0);

        AnchorPane.setTopAnchor(b, 20.0);
        AnchorPane.setLeftAnchor(b, 20.0);

        Scene s1 = new Scene(a, 200, 250);
        ps.setScene(s1);
        ps.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

Output:-



6) Tile Pane:

✓ Similar to FlowPane

✓ Arranges in rows

✓ Moves to next line when full

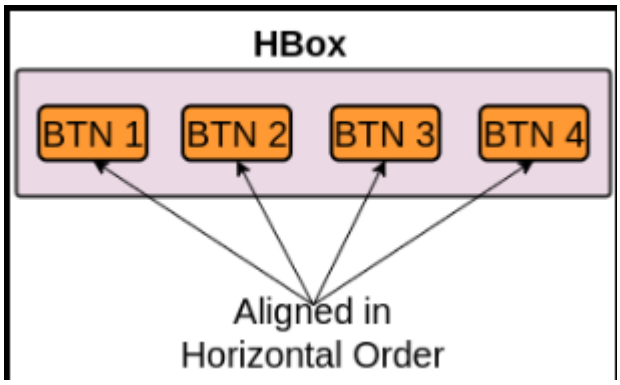
🔥 Difference:

- FlowPane → different sizes
- TilePane → **same size cells**

```
• ✓ Correct Example:
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.TilePane;

public class A extends Application
{
    public void start(Stage ps)
    {
        Button b1 = new Button("Ok");
        Button b2 = new Button("Cancel");
        Button b3 = new Button("Retry");
        Button b4 = new Button("Hide");
        Button b5 = new Button("HHH");

        b1.setMaxSize(Double.MAX_VALUE, Double.MAX_VALUE);
```

```
        b2.setMaxSize(Double.MAX_VALUE, Double.MAX_VALUE);
        b3.setMaxSize(Double.MAX_VALUE, Double.MAX_VALUE);
        b4.setMaxSize(Double.MAX_VALUE, Double.MAX_VALUE);
        b5.setMaxSize(Double.MAX_VALUE, Double.MAX_VALUE);

        TilePane t = new TilePane();
        t.getChildren().addAll(b1, b2, b3, b4, b5);

        Scene s1 = new Scene(t, 200, 250);
        ps.setScene(s1);
        ps.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

7 - 8) HBox vs VBox (JavaFX Layout)



🔷 HBox

✅ HBox means Horizontal Box

👉 It arranges items in one row (left → right)

✅ All elements are placed side by side

👉 Example:

- Button1 | Button2 | Button3

👉 Simple:
HBox = horizontal (side by side)

✅ Features:

- Layout direction → Horizontal

- Items go in single row
- Can set spacing between items

✅ Example:

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.control.Button;

public class A extends Application
{
    public void start(Stage s)
    {
        Button b1 = new Button("One");
        Button b2 = new Button("Two");
        Button b3 = new Button("Three");

        HBox h = new HBox(10); // spacing = 10
        h.getChildren().addAll(b1, b2, b3);

        Scene sc = new Scene(h, 200, 250);
        s.setScene(sc);
        s.show();
    }
}
```

🔷 VBox

✔️ VBox means Vertical Box

👉 It arranges items in one column (top → bottom)

✔️ All elements are placed one below another

👉 Example:

- Button1
- Button2
- Button3

👉 Simple:
VBox = vertical (up-down)

✔️ Features:

- Layout direction → Vertical
- Items go in single column

- Can set spacing between items

✅ Example:

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.scene.control.Button;

public class A extends Application
{
    public void start(Stage s)
    {
        Button b1 = new Button("One");
        Button b2 = new Button("Two");
        Button b3 = new Button("Three");

        VBox v = new VBox(10); // spacing = 10
        v.getChildren().addAll(b1, b2, b3);

        Scene sc = new Scene(v, 200, 250);
        s.setScene(sc);
        s.show();
    }
}
```

🧠 **Final Quick Revision Trick**

Pane	Meaning
FlowPane	Line by line
StackPane	On top
GridPane	Table
BorderPane	5 parts
AnchorPane	Fix position
TilePane	Same size grid

Note :- Anchor Pane code is very hard to understand

Border Pane	Anchor Pane
1) Anchor Pane allows the edges of child nodes to be anchored to an offset from the anchor pane's edges	1) Border Pane lays out children in top, left, right, bottom, and center positions.

Difference between Flow Pane and Tile Pane

Flow Pane	Tile Pane
1) The Tile Pane places all of the child nodes in a single row and pushes in the next line if row is completed but each cell have same size	1) The Tile Pane places all of the child nodes in a single row and pushes in the next line if row is completed but each cell have same size

BorderPane	AnchorPane
Layout in 5 areas	Fix position using edges
Top, Bottom, Left, Right, Center	Set distance from edges
Structured layout	Flexible positioning

FlowPane	TilePane
Items flow in rows	Items arranged in grid
Moves to next line automatically	Fixed rows & columns
Different sizes allowed	Same size for all

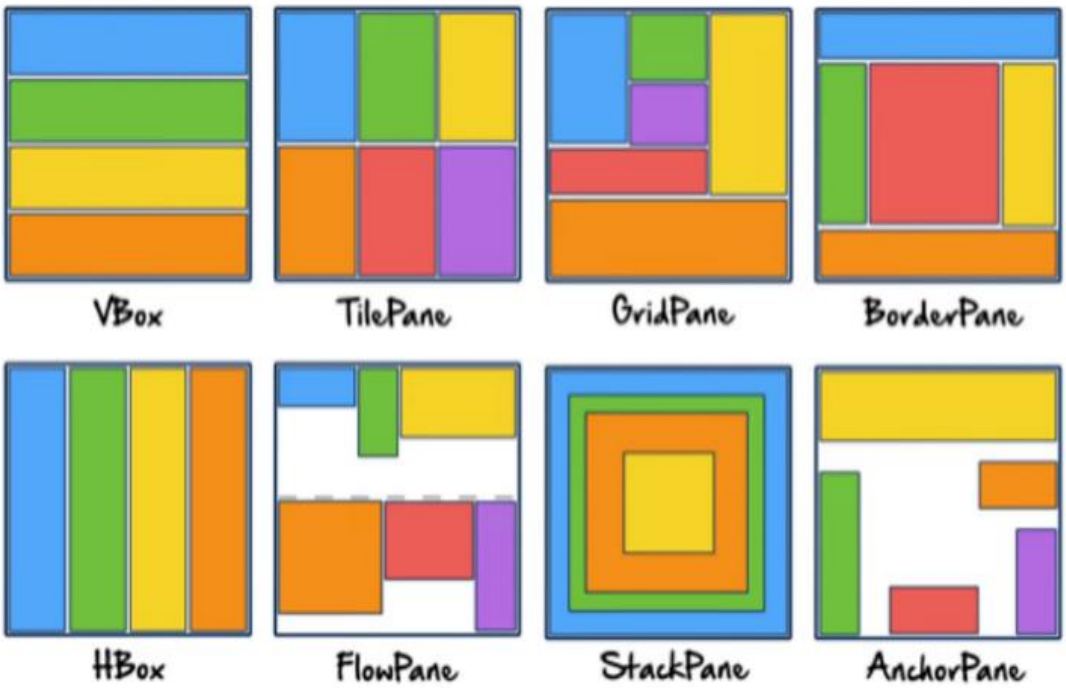
HBox	VBox
Horizontal layout	Vertical layout
Left → Right	Top → Bottom
One row	One column
Side by side items	One below another

🧠 **FINAL MEMORY TRICK**

👉 HBox = **H = Horizontal**
👉 VBox = **V = Vertical**

🔥 **One-Line Answer (Exam)**

👉 **HBox arranges nodes horizontally, while VBox arranges nodes vertically.**



Q-3 Explain UI controls and shapes

Ans:

UI Controls:

👉 UI Controls = **Elements used to interact with user**
Examples: Button, TextField, Label, etc.

✓ Label is used to **display text only**
✕ User cannot edit it

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.FlowPane;
```

```java
public class A extends Application
{
    public void start(Stage s)
    {
        Label l = new Label("Hello");

        FlowPane f = new FlowPane();
        f.getChildren().addAll(l);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

2) Labeled

✓ Labeled is a **parent class (abstract class)**

👉 It is used by:

- Label
- Button
- CheckBox
- RadioButton

✓ These controls can show:

- Text
- Image (graphics)

👉 Simple:
Labeled = Parent → Label, Button etc. are children

3) Button:

✓ Button is used to **perform action when clicked**

☑ **Example:**

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        Button b1 = new Button("Ok");
        Button b2 = new Button("Cancel");

        FlowPane f = new FlowPane();
        f.getChildren().addAll(b1, b2);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

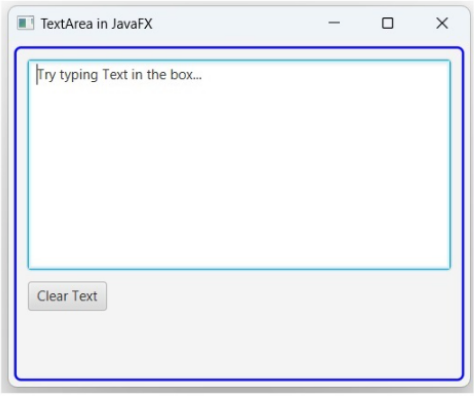
4) TextField:

✓ Used to enter **single line text**

✅ **Example:**

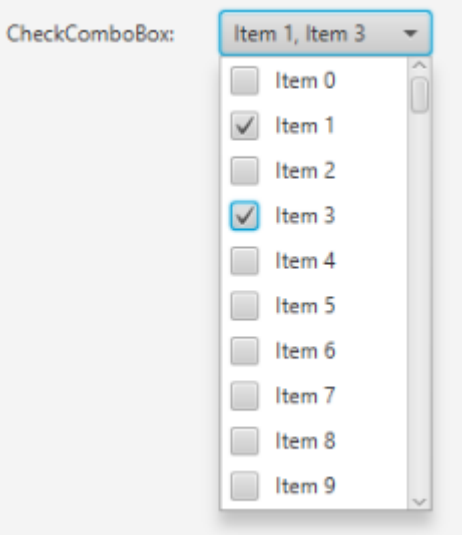
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.TextField;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        TextField t = new TextField();

        FlowPane f = new FlowPane();
        f.getChildren().addAll(t);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

5) TextArea:




```
✓ Used to enter multiple lines (like address) import
javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.TextArea;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        TextArea t = new TextArea();

        FlowPane f = new FlowPane();
        f.getChildren().addAll(t);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

6) CheckBox:



✓ Can be **true (checked) or false (unchecked)**

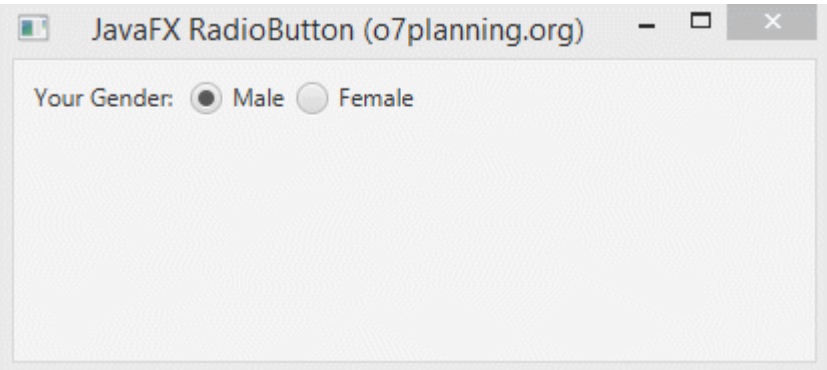
✓ User can select:

- One
- Multiple
- None

✅ **Example:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.CheckBox;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        CheckBox c1 = new CheckBox("Reading");
        CheckBox c2 = new CheckBox("Dancing");
        CheckBox c3 = new CheckBox("Singing");

        FlowPane f = new FlowPane();
        f.getChildren().addAll(c1, c2, c3);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

7) RadioButton:



✓ Only **one option can be selected**

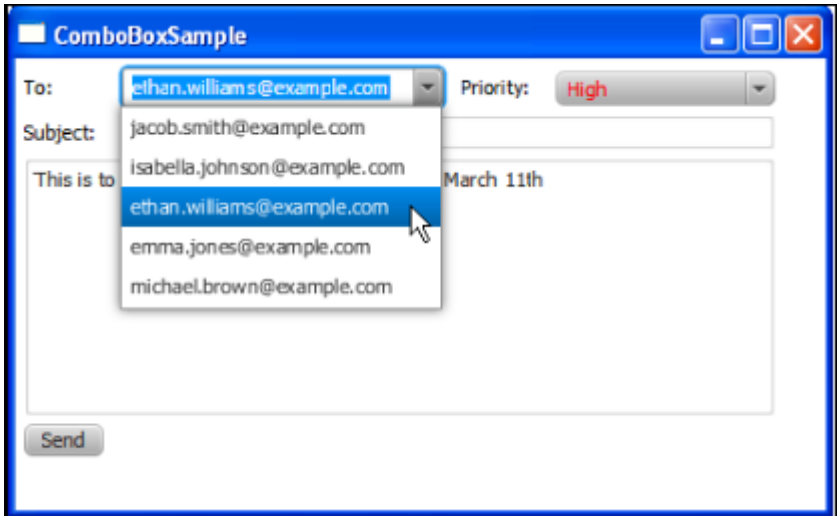
✓ Uses ToggleGroup

✅ **Example:**

```
         import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.RadioButton;
import javafx.scene.layout.FlowPane;
import javafx.scene.control.ToggleGroup;

public class A extends Application
{
```

```
    public void start(Stage s)
    {
        ToggleGroup t = new ToggleGroup();

        RadioButton r1 = new RadioButton("Male");
        r1.setToggleGroup(t);

        RadioButton r2 = new RadioButton("Female");
        r2.setToggleGroup(t);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(r1, r2);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

8) ComboBox:



✓ Shows **one item first**, others appear on click

👉 Difference:

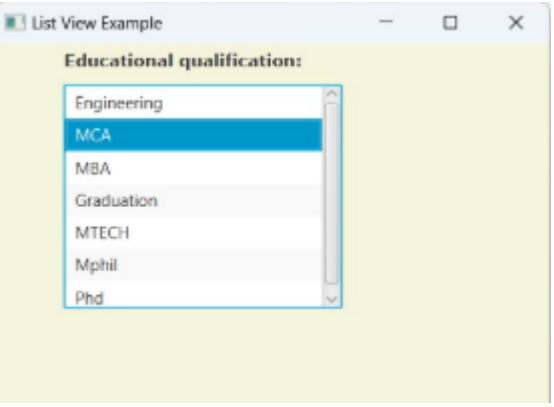
- ComboBox → show 1 item
- ListView → show all items

✅ **Example:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.ComboBox;
import javafx.scene.layout.FlowPane;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;

public class A extends Application
{
    public void start(Stage s)
    {
```

```
        ObservableList<String> items =
                FXCollections.observableArrayList("M.B.A", "M.Sc.IT", "I.T");

        ComboBox<String> c = new ComboBox<>(items);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(c);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

9) ListView:

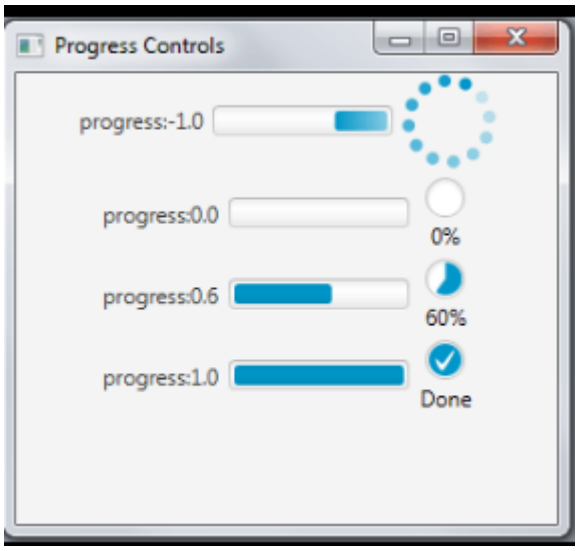


✓ Shows all items at once (scrollable list)

✅ Example:

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.ListView;
import javafx.scene.layout.FlowPane;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;

public class A extends Application
{
    public void start(Stage s)
    {
        ObservableList<String> items =
                FXCollections.observableArrayList("M.B.A", "M.Sc.IT", "I.T");

        ListView<String> l = new ListView<>(items);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(l);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

10) ProgressBar:



✓ **Shows task completion (0 to 1)**

☑ **Example:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.ProgressBar;
import javafx.scene.control.ProgressIndicator;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        ProgressBar p = new ProgressBar(0.6);
        ProgressIndicator pi = new ProgressIndicator(0.6);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(p, pi);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

11) Slider

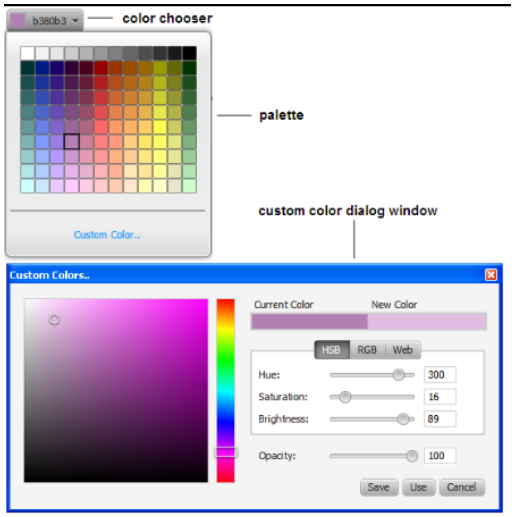
✓ **Select value by sliding**

☑ **Example:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Slider;
import javafx.scene.layout.FlowPane;

public class A extends Application
```

```
{
    public void start(Stage s)
    {
        Slider sl = new Slider();
        sl.setMin(0);
        sl.setMax(100);
        sl.setValue(50);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(sl);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

12) ColorPicker:



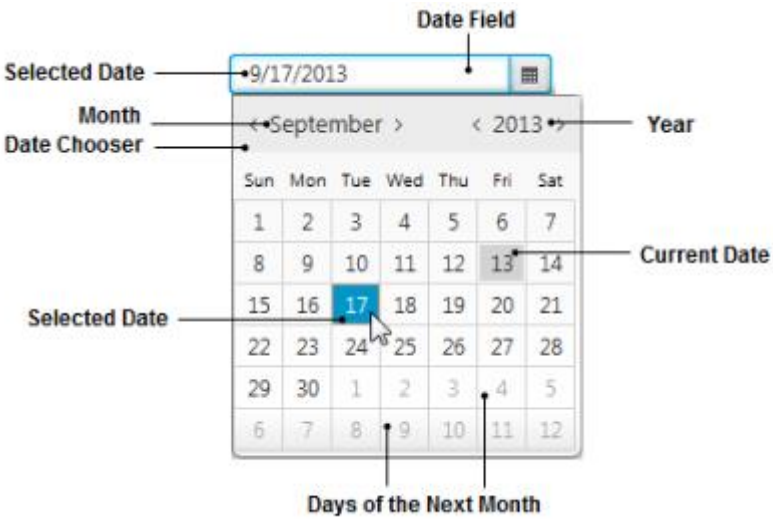
✓ Used to select a color

☑ Full Code:

```
        import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.ColorPicker;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        ColorPicker c = new ColorPicker();

        FlowPane f = new FlowPane();
        f.getChildren().addAll(c);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }

    public static void main(String[] args)
    {
```

```
        launch(args);
    }
}
```

13) DatePicker:

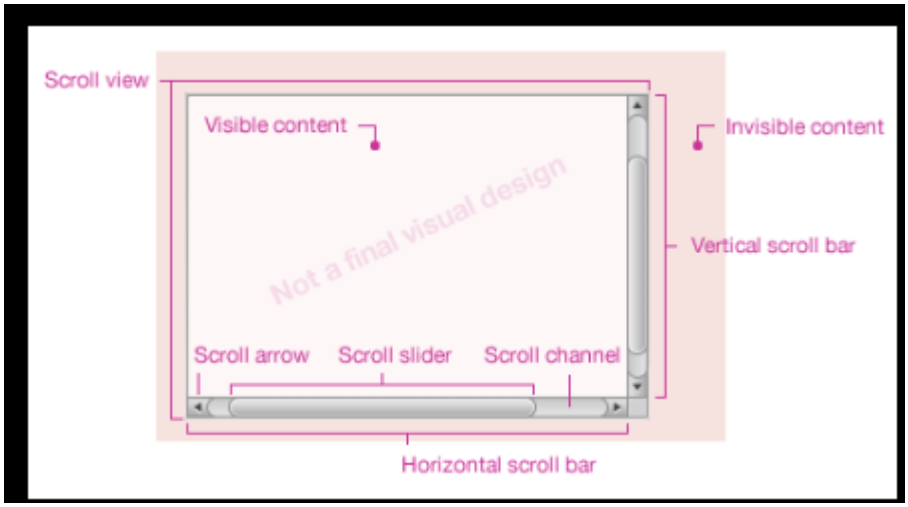


✓ **Used to select date from calendar**

✅ **Full Code:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.DatePicker;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        DatePicker d = new DatePicker();

        FlowPane f = new FlowPane();
        f.getChildren().addAll(d);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

14) ScrollBar:



✓ **Used to select value by scrolling**

☑ **Full Code:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.ScrollBar;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        ScrollBar sb = new ScrollBar();
        sb.setMin(0);
        sb.setMax(100);
        sb.setValue(50);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(sb);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

15) PasswordField:

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.PasswordField;
import javafx.scene.layout.FlowPane;

public class A extends Application
{
    public void start(Stage s)
    {
        PasswordField p = new PasswordField();

        FlowPane f = new FlowPane();
        f.getChildren().addAll(p);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }

    public static void main(String[] args)
    {
        launch(args);
    }
}
```

🧠 Quick Memory Tip

👉 Last 4 controls:

- **ColorPicker → color choose**

- DatePicker → date choose
- ScrollBar → scroll value
- PasswordField → hidden text

🧠 FINAL MEMORY SHORTCUT

👉 Group them:

Text Input:

- TextField (single line)
- TextArea (multi line)
- PasswordField

Selection:

- CheckBox (multiple)
- RadioButton (single)
- ComboBox
- ListView

Display:

- Label
- ProgressBar

Advanced:

- Slider
- ColorPicker
- DatePicker
- ScrollBar

Shapes:

✅ JavaFX Shapes (Simple Understanding)

👉 **Shapes are used to draw graphics like:**

- **Line**
- **Circle**
- **Rectangle**
- **Curve**
- **Polygon**

👉 **All shapes come from package:**

javafx.scene.shape

1) **Line**

✓ **Line means straight line between two points**

👉 **Class: Line**

👉 **Methods:**

- **setStartX() → starting X**
- **setStartY() → starting Y**
- **setEndX() → ending X**
- **setEndY() → ending Y**

✅ **Example:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Line;
import javafx.scene.Group;

public class A extends Application
{
    public void start(Stage s)
    {
        Line l = new Line();
        l.setStartX(100.0);
        l.setStartY(150.0);
        l.setEndX(500.0);
        l.setEndY(150.0);

        Group g = new Group(l);
        Scene s1 = new Scene(g, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

2) **Rectangle**

✓ **Rectangle = box shape**

👉 **Methods:**

- **setX() → position X**
- **setY() → position Y**
- **setWidth()**
- **setHeight()**

✅ **Example:**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Rectangle;
import javafx.scene.Group;

public class A extends Application
{
    public void start(Stage s)
    {
        Rectangle r = new Rectangle();
        r.setX(150.0);
        r.setY(75.0);
        r.setWidth(400.0);
        r.setHeight(150.0);

        Group g = new Group(r);
        Scene s1 = new Scene(g, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

3) **Rounded Rectangle**

✓ **Rectangle with round corners**

👉 **Extra methods:**

- **setArcWidth()**
- **setArcHeight()**

✅ **Example:**

```
Rectangle r = new Rectangle();
r.setX(150.0);
r.setY(75.0);
r.setWidth(400.0);
r.setHeight(150.0);
r.setArcWidth(30.0);
r.setArcHeight(20.0);
```

4) **Circle**

Circle = all points at equal distance from center

👉 **Methods:**

- **setCenterX()**
- **setCenterY()**
- **setRadius()**

✅ **Example:**

```
Circle c = new Circle();
c.setCenterX(300.0);
c.setCenterY(150.0);
c.setRadius(50.0);
```

5) **Ellipse**

   ✓ **Ellipse = oval shape**

   👉 **Methods:**

- **setCenterX()**
- **setCenterY()**
- **setRadiusX()**
- **setRadiusY()**

   ✅ **Example:**

```
Ellipse e = new Ellipse();
e.setCenterX(300.0);
e.setCenterY(150.0);
e.setRadiusX(150.0);
e.setRadiusY(75.0);
```

6) **Arc**

   ✓ **Arc = part of circle or ellipse**

   👉 **Methods:**

- **setCenterX()**
- **setCenterY()**
- **setRadiusX()**
- **setRadiusY()**
- **setStartAngle()**
- **setLength()**

   ✅ **Example:**

```
Arc a = new Arc();
a.setCenterX(400.0);
a.setCenterY(200.0);
a.setRadiusX(50.0);
a.setRadiusY(80.0);
a.setStartAngle(30.0);
a.setLength(70.0);
```

7) **PolyLine**

   ✓ **Same as polygon**

   ✕ **But NOT closed**

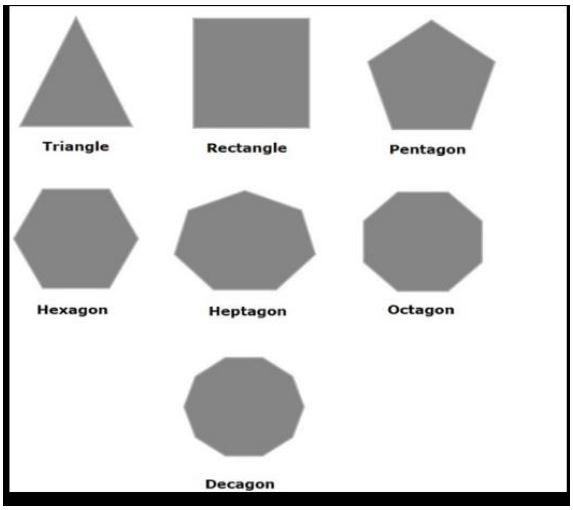
   👉 **Method:**

- **getPoints().addAll()**

   ✅ **Example:**

```
PolyLine p = new PolyLine();
p.getPoints().addAll(new Double[]{
    200.0,200.0, 300.0,100.0, 400.0,200.0
});
```

8) **Polygon (JavaFX)**



✓ Simple Meaning:

👉 Polygon = **closed shape made using lines**
👉 All lines are connected **end to end**
👉 Shape is **closed (no open ends)**

✓ Types of Polygon (based on sides)

👉 Number of sides → Name:

- **sides → Triangle**
- **sides → Quadrilateral**
  (Rectangle, Square, Parallelogram)
- **sides → Pentagon**
- **sides → Hexagon**
- **sides → Heptagon**
- **sides → Octagon**

✓ JavaFX Polygon Class

👉 Class name:
Polygon

👉 Package:
javafx.scene.shape

✓ Important Method

👉 Polygon has only one main method:

getPoints().addAll(...)

👉 **Used to add points (x, y)**

👉 **Each pair = one point**

👉 **Points automatically join → shape created**

```
✓ Triangle Example (3 points)
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Polygon;
import javafx.scene.Group;

public class A extends Application
{
    public void start(Stage s)
    {
        Polygon p = new Polygon();

        p.getPoints().addAll(
                200.0, 200.0,
                300.0, 100.0,
                400.0, 200.0
        );

        Group g = new Group(p);

        Scene s1 = new Scene(g, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

🧠 **Easy Understanding Trick**

👉 **Think like dots:**

(x1,y1) → (x2,y2) → (x3,y3) → back to start

✓ **JavaFX automatically connects last point to first → makes closed shape**

🔥 **One-Line Exam Answer**

👉 **Polygon is a closed shape formed by connecting multiple points, and in JavaFX it is created using the Polygon class with getPoints().addAll() method.**

```
☑ ▲ Triangle (3 sides)

p.getPoints().addAll(
    200.0, 200.0,
            300.0, 100.0,
            400.0, 200.0
);
```

```
✅ ■ Rectangle (4 sides)
p.getPoints().addAll(
    200.0, 100.0,
            400.0, 100.0,
            400.0, 300.0,
            200.0, 300.0
);

✅ ⬟ Pentagon (5 sides)
p.getPoints().addAll(
    300.0, 100.0,
            400.0, 180.0,
            360.0, 300.0,
            240.0, 300.0,
            200.0, 180.0
);

✅ ⬡ Hexagon (6 sides)

p.getPoints().addAll(
    250.0, 100.0,
            350.0, 100.0,
            400.0, 200.0,
            350.0, 300.0,
            250.0, 300.0,
            200.0, 200.0
);

✅ ⬢ Heptagon (7 sides)

p.getPoints().addAll(
    300.0, 100.0,
            380.0, 140.0,
            400.0, 220.0,
            350.0, 300.0,
            250.0, 300.0,
            200.0, 220.0,
            220.0, 140.0
);

✅ ⯃ Octagon (8 sides)

p.getPoints().addAll(
    260.0, 100.0,
            340.0, 100.0,
            400.0, 160.0,
            400.0, 240.0,
            340.0, 300.0,
            260.0, 300.0,
            200.0, 240.0,
            200.0, 160.0
);

✅ Decagon (10 sides)

p.getPoints().addAll(
    300.0, 100.0,
            360.0, 120.0,
            400.0, 180.0,
            400.0, 240.0,
            360.0, 300.0,
            300.0, 320.0,
            240.0, 300.0,
            200.0, 240.0,
            200.0, 180.0,
```

```
            240.0, 120.0
);
```

◈ 5 MARK QUESTIONS

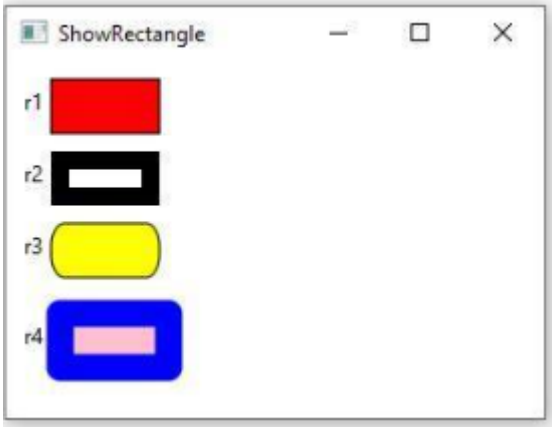
- **Write program to draw:**
  - **Line**
  - **Rectangle**
  - **Circle**
  - **Polygon**

👉 VERY HIGH CHANCE

Q-4: Color Class & Font Class

Ans:

🎨 Color Class



✓ Simple Meaning:

Color class is used to change the color of shapes, text, etc.

👉 By default:

- **All shapes are black**

👉 But JavaFX allows:

- **You can change to any color (Red, Blue, Green, etc.)**

- ✓ *Package: javafx.scene.paint*

✓ Important Point:

- **There is a base class called Paint**
- **Color class is a child of Paint**

✓ Methods:

- **setFill() → fill inside color**
- **setStroke() → border color**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Circle;
import javafx.scene.Group;
import javafx.scene.paint.Color;

public class A extends Application
{
```

```
    public void start(Stage s)
    {
        Circle c = new Circle();
        c.setCenterX(300.0);
        c.setCenterY(150.0);
        c.setRadius(50.0);

        c.setFill(Color.RED);    // change color

        Group g = new Group(c);
        Scene s1 = new Scene(g, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

🔤 Font Class

Font class is used to change text style

👉 You can change:

- **Font name (Arial, Calibri)**
- **Size**
- **Bold**
- **Italic**

- ✓ *Package: javafx.scene.text*

✓ Font is defined by:

- **Name**
- **Weight (Bold, Normal)**
- **Posture (Italic, Regular)**
- **Size**

✓ Classes used:

- **Font**
- **FontWeight**
- **FontPosture**

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
```

```java
import javafx.scene.layout.FlowPane;
import javafx.scene.text.Text;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.FontPosture;

public class A extends Application
{
    public void start(Stage s)
    {
        Text t = new Text("Hello how are you");

        Font f = Font.font("Calibri",
                FontWeight.EXTRA_BOLD,
                FontPosture.ITALIC,
                10);

        t.setFont(f);

        FlowPane f1 = new FlowPane();
        f1.getChildren().addAll(t);

        Scene s1 = new Scene(f1, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

Q-5: Property Binding

🔗 Property Binding (Very Simple)

Simple Meaning:
Property binding means:

👉 One object is connected to another object

👉 If source changes → target changes automatically

✓ Key Terms:

- **Source object → main value**
- **Target object → depends on source**

👉 Example:

- **If width changes → circle position also changes**

✓ Real Example:

👉 Problem:

- **Circle is not centered when window size changes**

👉 Solution:

- **Bind circle position with pane size**

✓ Important Concept:

```
target.bind(source)
```

```
✅ Example:

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Circle;

public class A extends Application
{
    public void start(Stage s)
    {
        Circle c = new Circle();
        Pane p = new Pane();

        // Binding (auto update position)
        c.centerXProperty().bind(p.widthProperty().divide(2));
        c.centerYProperty().bind(p.heightProperty().divide(2));

        c.setRadius(50.0);

        p.getChildren().addAll(c);

        Scene s1 = new Scene(p, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

🧠 FINAL MEMORY TRICK

👉 Color

- **Used to change color**
- **setFill(), setStroke()**

👉 Font

- **Used to style text**
- **Font name + size + style**

👉 Binding

- **Connect 2 objects**
- **Auto update**

Q-6: Image and ImageView Class

Ans:

✔ Simple Meaning:

- **Image → used to load image file**
- **ImageView → used to display image on screen**

👉 Flow:

Image → ImageView → Screen

✓ **Important Points:**

- **First create Image**
- **Then pass it to ImageView**
- **You can resize image without changing quality**

✓ **Methods:**

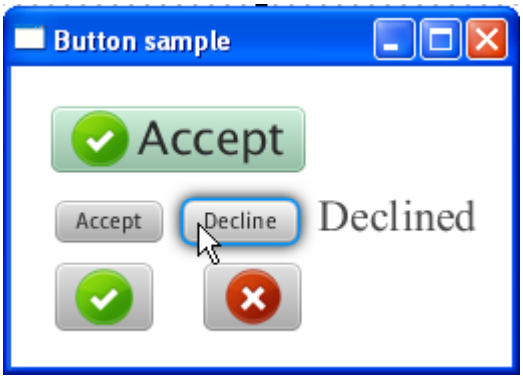
- **setX() → position X**
- **setY() → position Y**
- **setFitWidth() → width**
- **setFitHeight() → height**

✓ **Package:**

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

```
☑ Example:
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.Group;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import java.io.FileInputStream;

public class A extends Application
{
    public void start(Stage s) throws Exception
    {
        FileInputStream f = new FileInputStream("E:\\images\\1.jpg");

        Image i = new Image(f);
        ImageView v = new ImageView(i);

        v.setX(200.0);
        v.setY(150.0);
        v.setFitWidth(400.0);
        v.setFitHeight(300.0);

        Group g = new Group(v);
        Scene s1 = new Scene(g, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

Q-7: Event Handling in JavaFX



✓ Simple Meaning:
Event = user action
👉 Examples:

- Button click
- Mouse click
- Key press

✓ Package:

```
import javafx.event.Event;
import javafx.event.EventHandler;
```

✓ Types of Events:
1) Foreground Events
✓ Need user action
👉 Example: Button click
2) Background Events
✓ No user action
👉 Example: System process

✓ Event Components:

1. Event Source → where event happens (Button)
2. Event Target → where handled (Scene)
3. Event Type → type (click, press)

🔄 Event Flow
1) Route Construction
✓ Path created from Stage → Node
2) Capturing Phase

✓ Event goes top → bottom

3) Bubbling Phase

✓ Event goes bottom → top

✓ Event Handler:

✓ Code that runs when event happens

◈ Action Event (Button Click)

✓ Used for:

- Button click

✅ Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.GridPane;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.control.Label;

public class A extends Application
{
    public void start(Stage s)
    {
        Button b = new Button("Ok");
        Label l = new Label();

        GridPane g = new GridPane();

        EventHandler<ActionEvent> e = new EventHandler<ActionEvent>()
        {
            public void handle(ActionEvent e1)
            {
                l.setText("Hello");
            }
        };

        b.setOnAction(e);

        g.addRow(0, b);
        g.addRow(1, l);

        Scene s1 = new Scene(g, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

◈ Mouse Event

✓ Triggered by mouse

👉 Methods:

- setOnMousePressed
- setOnMouseReleased

- **setOnMouseClicked**
- **setOnMouseEntered**
- **setOnMouseExited**
- **setOnMouseDragged**

✅ **Example:**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;
import javafx.scene.input.MouseEvent;

public class A extends Application
{
    public void start(Stage s)
    {
        Label l = new Label();
        GridPane g = new GridPane();
        g.getChildren().addAll(l);

        Scene s1 = new Scene(g, 200, 250);

        s1.setOnMousePressed(e -> l.setText("Mouse Pressed"));
        s1.setOnMouseReleased(e -> l.setText("Mouse Released"));
        s1.setOnMouseClicked(e -> l.setText("Mouse Clicked"));
        s1.setOnMouseEntered(e -> l.setText("Mouse Entered"));
        s1.setOnMouseExited(e -> l.setText("Mouse Exited"));
        s1.setOnMouseDragged(e -> l.setText("Mouse Dragged"));

        s.setScene(s1);
        s.show();
    }
}
```

🔷 **Key Event**

✔ **Triggered by keyboard**

👉 **Methods:**

- **setOnKeyPressed**
- **setOnKeyReleased**
- **setOnKeyTyped**

```
✅ Example: import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;

public class A extends Application
{
    public void start(Stage s)
    {
        Label l = new Label();
```

```
        GridPane g = new GridPane();
        g.getChildren().addAll(l);

        Scene s1 = new Scene(g, 200, 250);

        s1.setOnKeyPressed(e -> l.setText("Key Pressed"));
        s1.setOnKeyReleased(e -> l.setText("Key Released"));
        s1.setOnKeyTyped(e -> l.setText("Key Typed"));

        s.setScene(s1);
        s.show();
    }
}
```

Q-8: Nested and Inner Classes

✓ Simple Meaning:

Class inside another class

✓ Types:

1. **Static Nested Class**

   ✓ No access to outer class variables

2. **Non-static Inner Class**

   ✓ Can access all outer class variables

3. **Local Class**

   ✓ Defined inside method

4. **Anonymous Class**

   ✓ No name, used directly

✅ Anonymous Example:

```
b.setOnAction(new EventHandler<ActionEvent>()
{
    public void handle(ActionEvent e)
    {
        System.out.println("button clicked");
    }
});
```

✅ Q-9: Animation

✓ Simple Meaning:

Animation = moving or changing object over time

👉 Package:

javafx.animation

✓ Simple Meaning:

Animation means adding movement or effects to objects like:

- Text
- Images
- Shapes

  👉 Example:
- Fade in/out
- Move object
- Change color
- Rotate

```
◈ 1) FadeTransition

        ✓ Used to change visibility (opacity)

        👉 Example: fade in / fade out

        ☑ Example:
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.layout.FlowPane;
import javafx.scene.shape.Circle;
import javafx.animation.FadeTransition;
import javafx.scene.paint.Color;
import javafx.util.Duration;

public class A extends Application
{
    public void start(Stage s)
    {
        Circle c = new Circle();
        c.setRadius(50.0);
        c.setFill(Color.RED);

        FlowPane f = new FlowPane();

        FadeTransition f1 = new FadeTransition(Duration.millis(1000));
        f1.setNode(c);
        f1.setFromValue(1.0);
        f1.setToValue(0.3);
        f1.setCycleCount(50);
        f1.setAutoReverse(false);
        f1.play();

        f.getChildren().addAll(c);

        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

2) FillTransition

✓ Used to change color

👉 Example: Blue → Green

✅ **Example:**

```
import javafx.animation.FillTransition;

FillTransition f1 = new FillTransition(Duration.millis(1000));
f1.setShape(c);
f1.setFromValue(Color.BLUE);
f1.setToValue(Color.GREEN);
f1.setCycleCount(50);
f1.setAutoReverse(false);
f1.play();
```

3) RotateTransition

✓ **Used to rotate object**

✅ **Example:**

```
import javafx.animation.RotateTransition;

RotateTransition f1 = new RotateTransition(Duration.millis(1000));
f1.setNode(p);
f1.setByAngle(360);
f1.setCycleCount(50);
f1.setAutoReverse(false);
f1.play();
```

4) ScaleTransition

✓ **Used to increase or decrease size**

✅ **Example:**

```
import javafx.animation.ScaleTransition;

ScaleTransition f1 = new ScaleTransition(Duration.millis(1000));
f1.setNode(c);
f1.setByX(3.5);
f1.setByY(3.5);
f1.setCycleCount(50);
f1.setAutoReverse(false);
f1.play();
```

5) StrokeTransition

✓ **Used to change border color**

✅ **Example:**

```
import javafx.animation.StrokeTransition;

StrokeTransition f1 = new StrokeTransition(Duration.millis(1000));
f1.setShape(c);
f1.setFromValue(Color.BLACK);
f1.setToValue(Color.GREEN);
f1.setCycleCount(50);
f1.setAutoReverse(true);
f1.play();
```

◈ **6) TranslateTransition**

✓ **Used to move object**

✅ **Example:**

```
import javafx.animation.TranslateTransition;

TranslateTransition f1 = new TranslateTransition(Duration.millis(1000));
f1.setNode(c);
f1.setByX(600.0);
f1.setCycleCount(50);
f1.setAutoReverse(false);
f1.play();
```

◈ **7) SequentialTransition**

✓ **Run animations one after another**

✅ **Example:**

```
SequentialTransition st = new SequentialTransition(c, ft, f1);
st.play();
```

◈ **8) PauseTransition**

✓ **Used to pause animation**

✅ **Example:**

```
PauseTransition pt = new PauseTransition();
pt.setDuration(Duration.millis(1000));
```

◈ **9) ParallelTransition**

✓ **Run animations together at same time**

✅ **Example:**

```
ParallelTransition st = new ParallelTransition(c, ft, f1);
st.play();
```

Q-10 Write a short note on Video and Audio or Multimedia

Ans:

Multimedia means playing video and audio in JavaFX

👉 JavaFX gives special classes to:

- Load media
- Control media

- Show media

✅ Main Classes

1) Media

✔ Represents media file (video or audio)

✔ Used to load file

---

2) MediaPlayer

✔ Used to control media

👉 It can:

- Play
- Pause
- Stop
- Control volume
- Set speed
- Loop video

---

3) MediaView

✔ Used to display video on screen

---

4) AudioClip (extra)

✔ Used for simple sound effects

🔁 Flow (Very Important)

File → Media → MediaPlayer → MediaView → Screen

✅ Important Conversion Methods

👉 These are used to convert file path:

- m.toURI() → convert file to URI
- .toURL() → convert URI to URL
- .toString() → convert URL to String

👉 JavaFX needs String format path

```
✅ Example (Corrected + Simple)
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.layout.FlowPane;
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
```

```java
import javafx.scene.media.MediaView;
import java.io.File;

public class A extends Application
{
    public void start(Stage s)
    {
        File m = new File("C:/Users/HP/Downloads/2.mp4");

        Media m1 = new Media(m.toURI().toString());
        MediaPlayer m2 = new MediaPlayer(m1);
        MediaView mv = new MediaView(m2);

        FlowPane f = new FlowPane();
        f.getChildren().addAll(mv);

        Scene s1 = new Scene(f, 400, 300);
        s.setScene(s1);
        s.show();

        m2.play(); // start video
    }
}
```

🧠 FINAL MEMORY TRICK

👉 Remember like this:

- Media = file
- MediaPlayer = control
- MediaView = display

👉 Flow:

Media → MediaPlayer → MediaView

👉 Extra:

- AudioClip → sound only

Q-11 Write a short note on Listener for observable object

Ans:

✓ Simple Meaning:

👉 **Listener = a watcher**

✓ It watches a value

✓ When value changes → it automatically runs code

✓ Observable Object:

👉 **Observable means value that can change**

👉 **Examples:**

- **StringProperty**
- **IntegerProperty**

- **BooleanProperty**

  ✓ These values can be observed (tracked)

✅ ChangeListener

✓ Used to detect change in ONE value

👉 When value changes → method runs

✓ Method:

void changed(ObservableValue<? extends T> observable, T oldValue, T newValue);

👉 Meaning:

- **observable → the value being watched**
- **oldValue → previous value**
- **newValue → new value**

✅ Why we use Listener?

👉 To automatically respond when value changes

✓ Example:

- **Value changes → print message**
- **Slider changes → update label**
- **Text changes → validate input**

```
✅ Example (Simple + Clear)
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.layout.FlowPane;
import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.beans.property.IntegerProperty;
import javafx.beans.property.SimpleIntegerProperty;

public class A extends Application
{
    public void start(Stage s)
    {
        // create observable value
        IntegerProperty n = new SimpleIntegerProperty(10);

        // add listener
        n.addListener(new ChangeListener<Number>()
        {
            public void changed(ObservableValue<? extends Number> observable,
                                Number oldValue,
                                Number newValue)
            {
                System.out.println("Old value: " + oldValue +
                        ", New value: " + newValue);
            }
        });
```

```
        // change value (this will trigger listener)
        n.set(20);

        FlowPane f = new FlowPane();
        Scene s1 = new Scene(f, 200, 250);
        s.setScene(s1);
        s.show();
    }
}
```

🧠 **Step-by-Step Flow**

1. **Create value → n = 10**
2. **Add listener**
3. **Change value → n.set(20)**
4. **Listener runs automatically**

🧠 **FINAL MEMORY TRICK**

👉 **Listener = watcher**

👉 **Observable = changeable value**

👉 **When value changes:**
Old value → New value → Listener runs

🔥 **One-Line Exam Answer:**

👉 **Listener is used to detect and respond automatically when observable value changes**